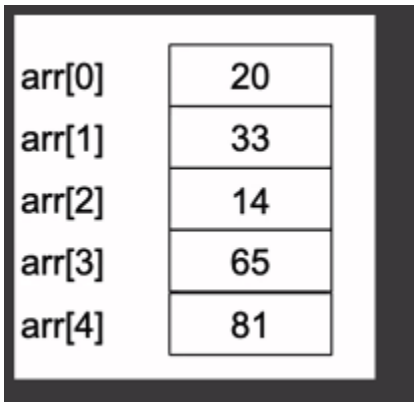
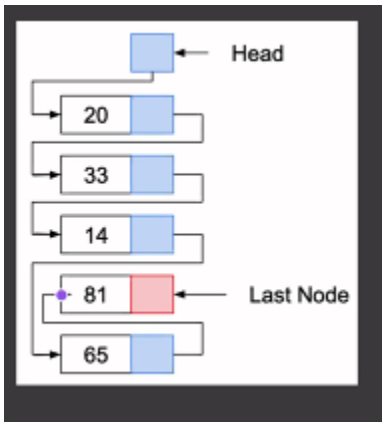
Darren Lee
CSE13S

Week 4 Notes

4/19/21
Stacks and Queues

- Arrays
- 

    - Allow random access
    - Each element can be accessed directly in constant time
- Linked lists
- 

    - Allow sequential access
    - Each element can only be accessed in a particular order

Stacks and Queues

- Operate as containers for other data types
    - For the most part, data of any type may be placed in a stack or a queue
- Both have:
    - Well-defined semantics
    - A fixed set of operations
    - A distinct ordering to their elements

Stacks

- Objects are added and remove using last-in-first-out (LIFO) order
- Capacity of a stack is the number of stack elements that will fit
- This capacity can be increased as number of elements increases through reallocation (dynamic)
- Often implemented with arrays, but other possibilities exist as long as the semantics are preserved

```c
#ifndef __STACK_H__
#define __STACK_H__

#include <stdbool.h>
#include <stdint.h>

typedef uint32_t item;

typedef struct Stack {
    uint32_t top;
    uint32_t capacity;
    item *items;
} Stack;

Stack *stack_create(void);

void stack_delete(Stack **s);

bool stack_empty(Stack *s);

bool stack_full(Stack *s);

bool stack_push(Stack *s, item i);

bool stack_pop(Stack *s, item *i);
```

-

stack_create()

```c
#define MIN_CAPACITY 16

Stack *stack_create(void) {
    Stack *s = (Stack *)malloc(sizeof(Stack));
    if (s != NULL) {
        s->top = 0;
        s->capacity = MIN_CAPACITY;
        s->items = (item *)malloc(s->capacity * sizeof(item));
        if (s->items == NULL) {
            free(s);
            s = NULL;
        }
    }
    return s;
}
```

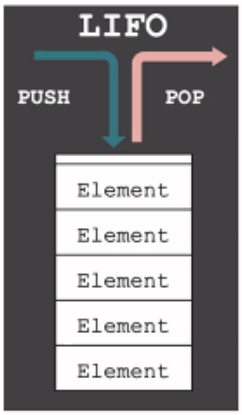
stack_full()

```
bool stack_full(Stack *s) {
    return s->top == s->capacity;
}
```

stack_empty()

```
bool stack_empty(Stack *s) {
    return s->top == 0;
}
```

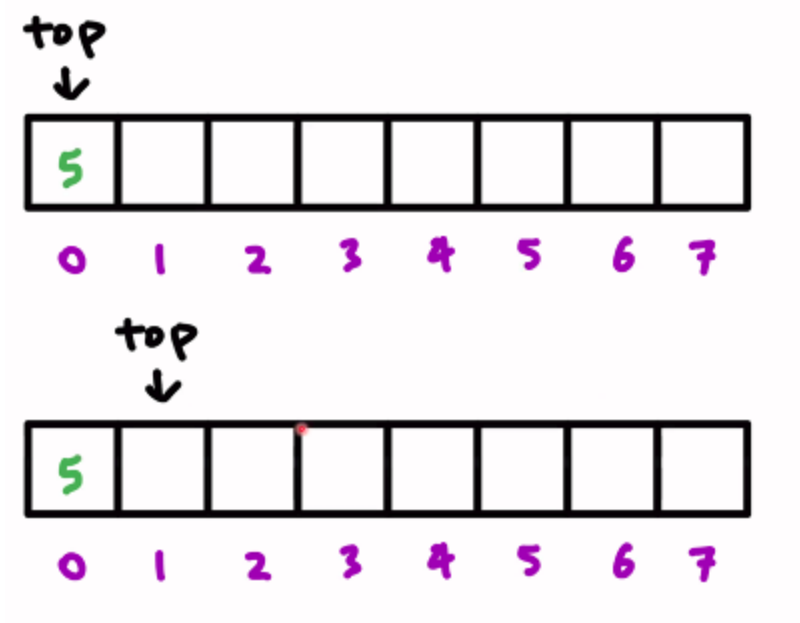
stack_push()

- Add an element to the top of a stack
- Example is for a dynamic stack, in which memory is reallocated when the stack is “full”
- 


```
bool stack_push(Stack *s, item i) {
    if (stack_full(s)) {
        s->capacity *= 2;
        s->items = (item *)realloc(s->items, s->capacity * sizeof(item));
    }
    s->items[s->top] = i;
    s->top += 1;
    return true;
}
```

Example: pushing items onto a stack

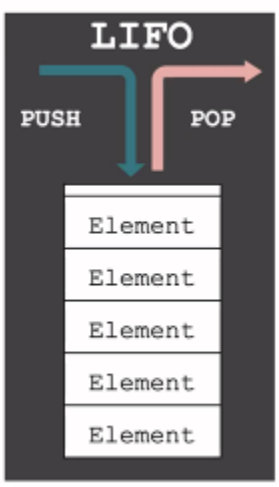
- The top of the stack is the index of the next available spot



-
- Assign the value, move the top forward

stack_pop()

- Remove the element at the top of a stack
- Returns false if the stack is empty




```
bool stack_pop(Stack *s, item *i) {
    if (stack_empty(s)) {
        return false;
    }
    s->top -= 1;
    *i = s->items[s->top];
    return true;
}
```

-

Stacks using linked lists

- Stacks can also be implemented using linked lists
- Each element of the stack points to the next
- The internal stack in memory is not linked

```
#ifndef __STACK_H__
#define __STACK_H__

#include <stdbool.h>
#include <stdint.h>

typedef uint32_t item;

typedef struct Node Node;

struct Node {
    Node *next;
    item data;
};

typedef struct Stack {
    Node *top;
} Stack;

Stack *stack_create(void);

void stack_delete(Stack **s);

bool stack_empty(Stack *s);

bool stack_push(Stack *s, item i);

bool stack_pop(Stack *s, item *i);

#endif
```

-

stack_create()

```
Stack *stack_create(void) {
    Stack *s = (Stack *)malloc(sizeof(Stack));
    if (s != NULL) {
        s->top = NULL;
    }
    return s;
}
```

stack_push()

```
static Node *node_create(item i) {
    Node *n = (Node *)malloc(sizeof(Node));
    if (n) {
        n->data = i;
        n->next = NULL;
    }
    return n;
}

bool stack_push(Stack *s, item i) {
    Node *n = node_create(i);
    n->next = s->top;
    s->top = n;
    return true;
}
```

stack_pop()

```
static void node_delete(Node **n) {
    free(*n);
    *n = NULL;
}

bool stack_pop(Stack *s, item *i) {
    if (stack_empty(s)) {
        return false;
    }
    *i = s->top->data;
    Node *n = s->top;
    s->top = s->top->next;
    node_delete(&n);
    return true;
}
```

Queue.h

- A queue is an abstract data type (ADT), so let's define it
- We need:
    - A head, a tail, and a size
    - A pointer to an array

- A constructor
- A destructor
- Empty and full predicates
- Enqueue - inserts into the queue
- Dequeue - removes from the queue

```
#ifndef _QUEUE_H
#define _QUEUE_H

#include <stdbool.h>
#include <stdint.h>

typedef uint32_t item;

typedef struct queue {
    uint32_t head;
    uint32_t tail;
    uint32_t size;
    item *Q;
} queue;

queue *newQueue(uint32_t);

void delQueue(queue *);

bool emptyQ(queue *);

bool fullQ(queue *);

bool enqueue(queue *, item *);

bool dequeue(queue *, item *);
#endif
```

-

```
queue *newQueue(uint32_t size) {
    queue *q = (queue *) malloc(sizeof(queue));
    if (q) {
        q->head = q->tail = 0;
        q->size = size;
        q->Q = (item *) calloc(size, sizeof(item));
        if (q->Q) {
            return q;
        }
        free(q);
    }
    return (queue *) 0;
}
```

Create a Queue

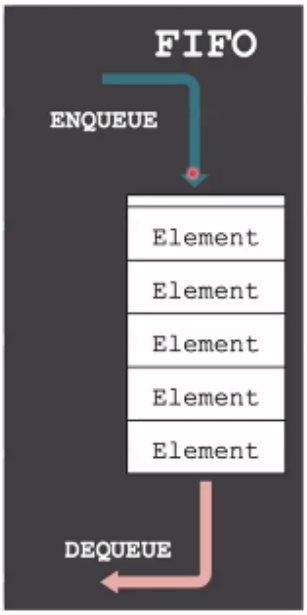
Delete a queue

Delete a Queue

```
void delQueue(queue *q) {
    if (q) {
        free(q->Q);
        free(q);
    }
}
```

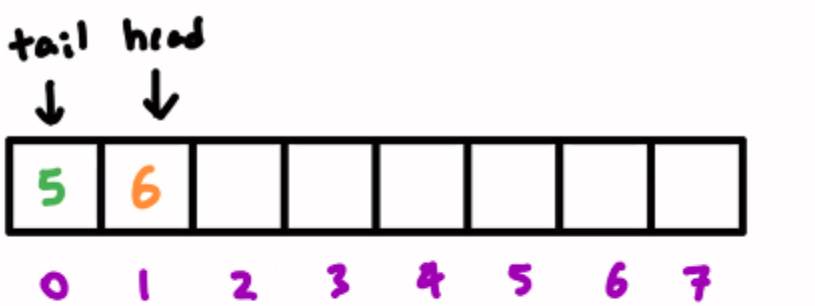
Enqueue an item

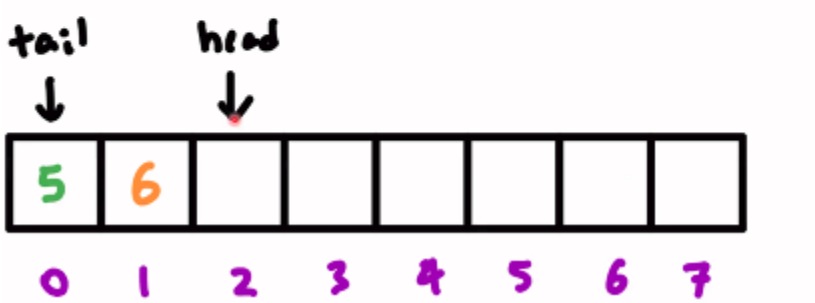
```
bool enqueue(queue *q, item *i) {
    if (q) {
        if (fullQ(q)) {
            return false;
        }
        q->Q[q->head] = *i;
        q->head = succ(q, q->head);
        return true;
    } else {
        return false;
    }
}
```



Ex: enqueueing items

- If head and tail point to the same place, it means the queue is empty





- Insert at the head, move the head forward
- Pass back the item at the tail, move the tail forward

```
bool emptyQ(queue *q) {
    if (q) {
        return q->head == q->tail;
    }
    return true;
}

bool fullQ(queue *q) {
    if (q) {
        return succ(q, q->head) == q->tail;
    }
    return true;
}
```

- We define an empty queue to have the head and tail indicating the same slot
- How do we define full
    - We sacrifice one queue slot, and say it is full if:
    - head+1=tail(mod n)

Unbounded queues

- Can be implemented with a linked list
- The unbounded queue itself is a wrapper for a linked list
- The wrapper keeps track of the head and tail

-
```c
#ifndef __QUEUE_H__
#define __QUEUE_H__

#include <stdint.h>
#include <stdbool.h>

typedef uint32_t item;

typedef struct Node Node;

struct Node {
    Node *next;
    item data;
};

typedef struct Queue {
    Node *head;
    Node *tail;
} Queue;

Queue *queue_create(void);

void queue_delete(Queue **q);

bool queue_empty(Queue *q);

bool enqueue(Queue *q, item i);

bool dequeue(Queue *q, item *i);

#endif
```

queue_create()

- Both the head and tail are NULL to begin with
-
```c
Queue *queue_create(void) {
    Queue *q = (Queue *)malloc(sizeof(Queue));
    q->head = q->tail = NULL;
    return q;
}
```

enqueue()

- First, create the node to enqueue
- If the tail is NULL, then the queue is empty, and so the tail and head should be the created node
- Otherwise, add the node at the end and update the tail

```
static Node *node_create(item i) {
    Node *n = (Node *)malloc(sizeof(Node));
    if (n) {
        n->data = i;
        n->next = NULL;
    }
    return n;
}

bool enqueue(Queue *q, item i) {
    Node *n = node_create(i);
    if (!n) {
        return false;
    }

    if (q->tail == NULL) {
        q->tail = q->head = n;
        return true;
    }

    q->tail->next = n;
    q->tail = n;
    return true;
}
```

-

dequeue()

- First, save the value of the node being dequeued
- The head of the queue is moved forward one node
- We save a pointer to it so we can free the memory it pointed to

Differences between stacks and queues

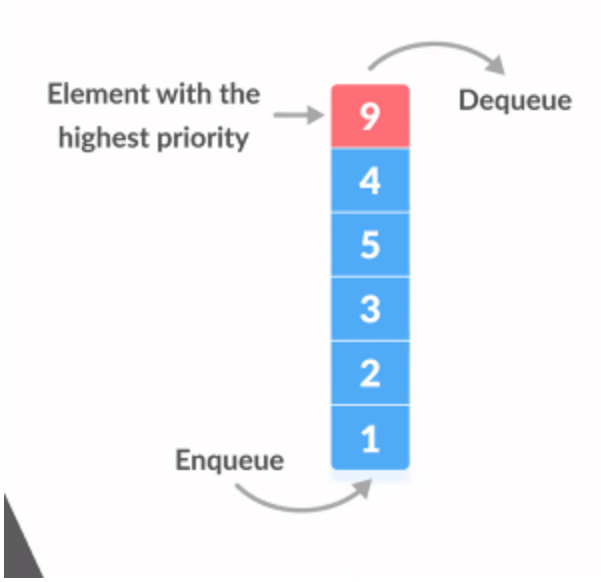
- Removing
    - Stack removes the most recent
    - Queue removes the least recent

Queue implementations

- Commonly implemented using vert-based data structures

Priority queue

- Like a generic queue, but every element has a priority associated with it
- Element of highest priority is dequeued before elements with lower priority
- In the case of two elements of the same priority, preference is given to the position of the element in the queue

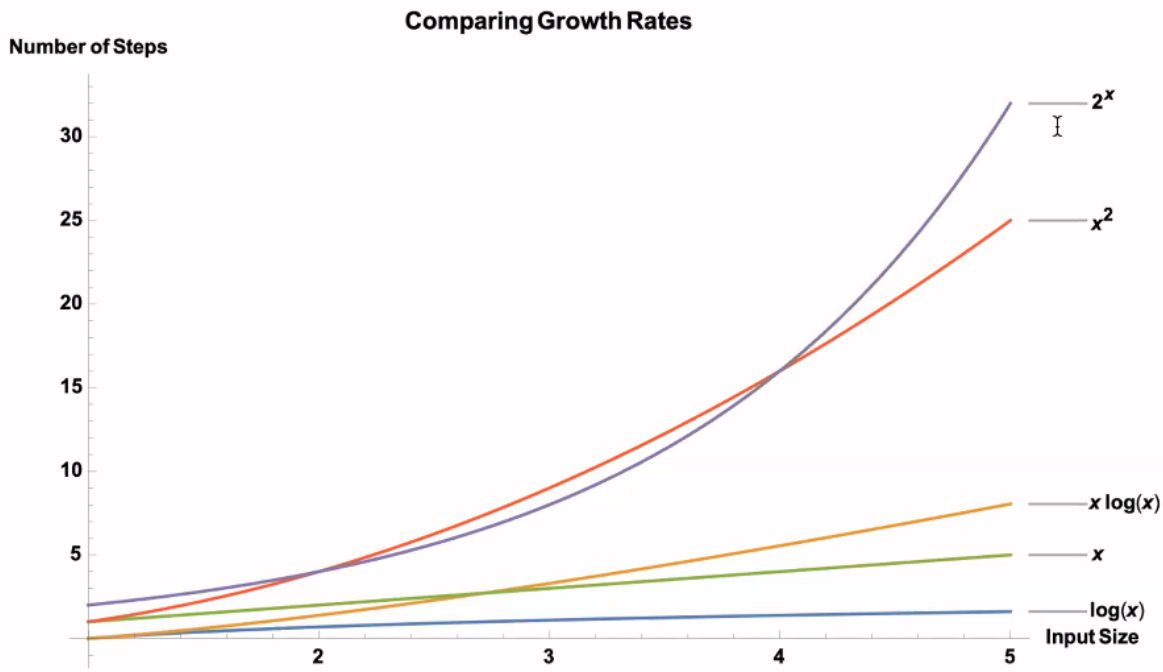
- 

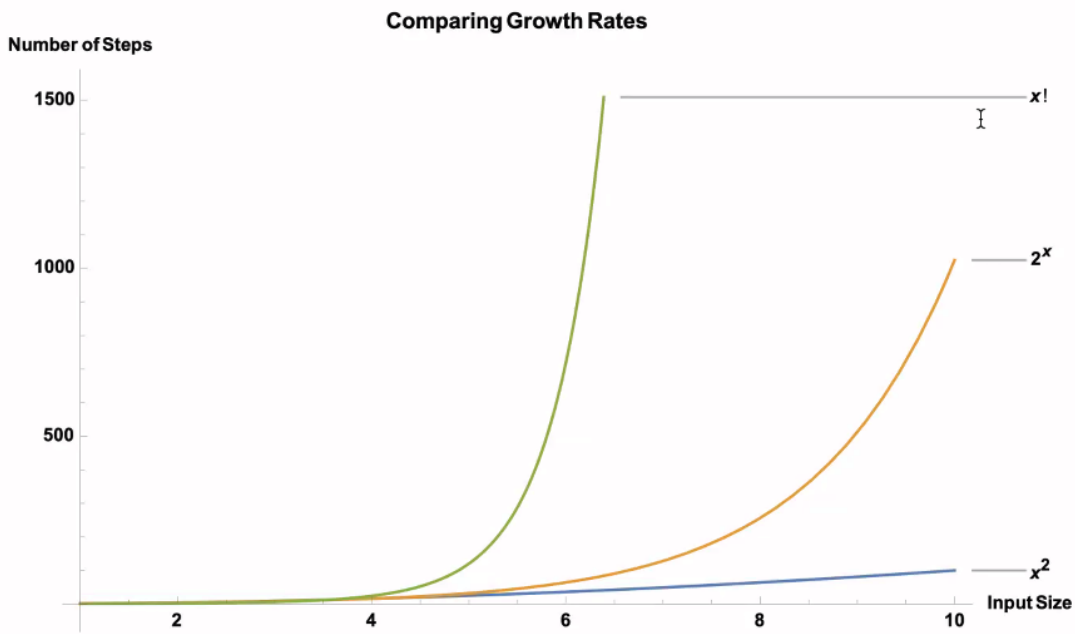
Summary
- Stacks are LIFO
- Queues are FIFO
- Both can be implemented with arrays and linked lists

4/21/21
Computational Complexity
- We talk about the size of a problem
    - How long is the string?
    - How large is the number?
    - How many items in the set?
- We often talk about the size of the input (ie length of a string)
- We talk about cardinality and magnitude

Some common functions

function	10	30
log(x)	2.30259	3.4012
x	10	30
x log(x)	23.0259	102.036
x^2	100	900
x^3	1000	27,000
2^x	1024	1,073,741,824
x!	3,628,800	265,252,859,812,191,058,636,308,480,000,000





Exemplars

Order	Example
$O(\log(n))$	Binary Search
$O(n)$	Find Minimum
$O(n\log(n))$	Merge Sort
$O(n^2)$	Bubble Sort
$O(n^3)$	Matrix Multiply
$O(2^n)$	Enumerate Subsets
$O(n!)$	Enumerate Permutations

O(1) is referred to as constant time

Big O is worst case, big Omega is best case, theta is average case

Formal mathematics

$$f(n) \in O(g(n)) \text{ if and only if } \exists c, n_0 \text{ such that } \forall n > n_0, f(n) \leq c \cdot g(n)$$

Formalities

- When we say “Bubble Sort is O(n^2)” or “Bubble Sort is order n^2” we’re not being very precise:
  - Every computer scientist should know what we mean
  - But we must agree on what we mean
- Once x gets large enough, there is some constant c where f(x) is always bounded by c*g(x)
  - f(n)<=c*g(n)
- $$O = \{f : \text{there exist } n_0 \text{ and } c \text{ such that } 0 \leq f(n) \leq c \cdot g(n) \text{ for all } n > n_0\}$$

Simple linear algorithm O(n)

```
int minIndex(uint32_t a[], int first, int last)
{
  int min = first;
  for (int i = first; i < last; i += 1) {
    min = a[i] < a[min] ? i : min;
  }
  return min;
}
```

$O(n)$

Simple quadratic algorithm O(n^2)

```
void insertionSort(uint32_t a[], int length) {
  for (int i = 1; i < length; i += 1) {
    int j = i;
    uint32_t tmp = a[i];
    while (j > 0 && a[j - 1] > tmp) {
      a[j] = a[j - 1];
      j -= 1;
    }
    a[j] = tmp;          O(n)
  }
  return;                O(n)
}
```

A less simple algorithm
On average O(n log(n)) Worst case $O(n^2)$

```
int partition(uint32_t a[], int32_t low, int32_t high) {
  uint32_t pivotValue = a[(low + high) / 2];

  int32_t i = low - 1;
  int32_t j = high + 1;
  do {
    do {
      i += 1;
    } while (a[i] < pivotValue);
    do {
      j -= 1;
    } while (a[j] > pivotValue);
    if (i < j) {
      SWAP(a[i], a[j]);
    }
  } while (i < j);
  return j;
}

void quickSort(uint32_t a[], int32_t low, int32_t high) {
  if (low < high) {
    uint32_t p = partition(a, low, high);

    quickSort(a, low, p);
    quickSort(a, p + 1, high);
  }
  return;
}
```

What about these?

$O(n)$

It all depends on this!

$O(\log(n))$ or $O(n)$

Estimating complexity

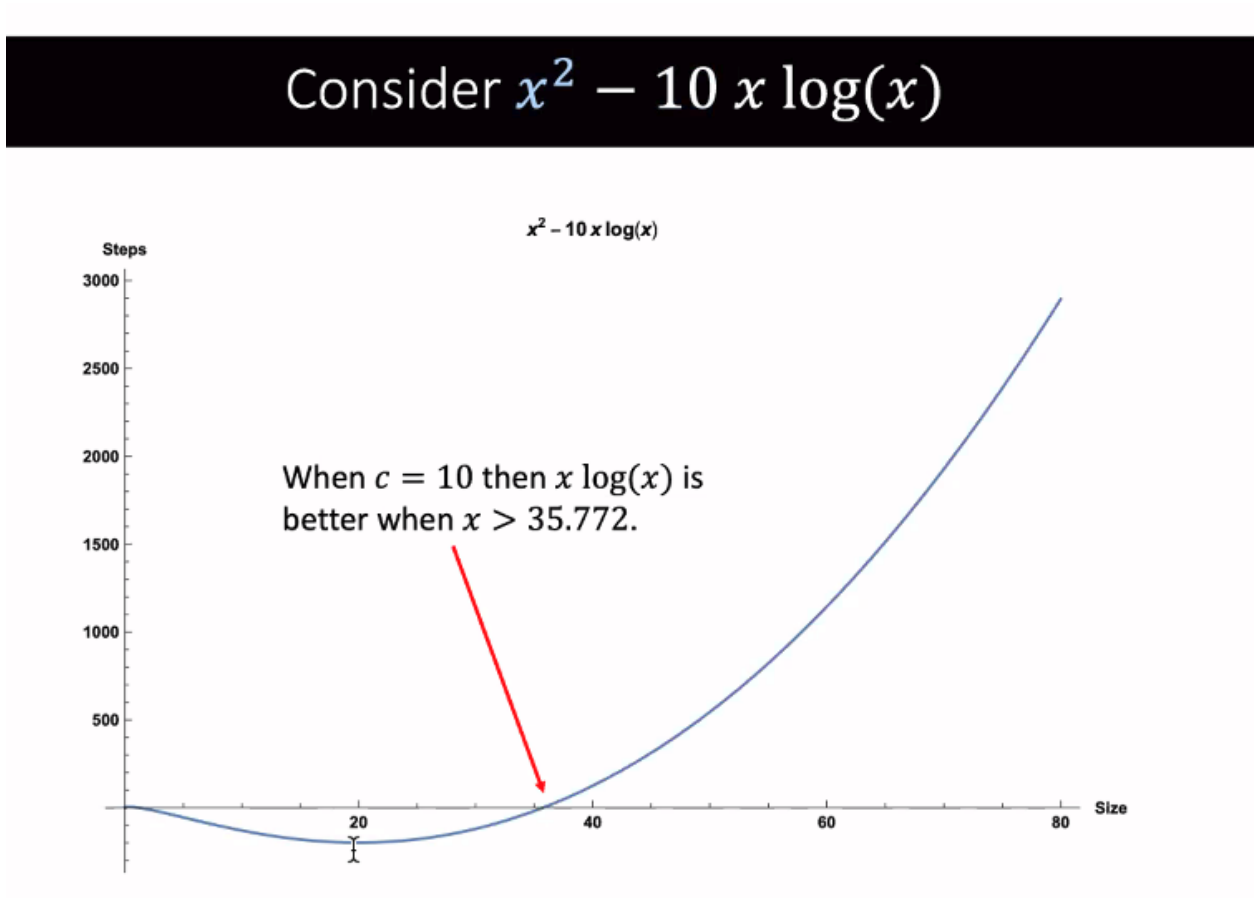
- For now, we can adopt some simple rules for estimating complexity
- We do it by looking at the loops and the recursion
- We add the complexity of loops at the same level
    - In general, this does not matter for large inputs
    - Just affects constant time c

So what about c?

- You can think of c as the overhead that an algorithm requires
    - For small n, bubble sort is faster than quicksort because of the overhead
    - We say that quicksort has a larger constant or larger c
- Optimizing your code will make c smaller
    - It may make a big difference in the run-time of your program
    - But it is only a constant speed-up

- Changing to a better algorithm is much more impactful
- The existence of an efficient algorithm determines whether you can solve a particular problem

Consider x^2-10xlog(x)

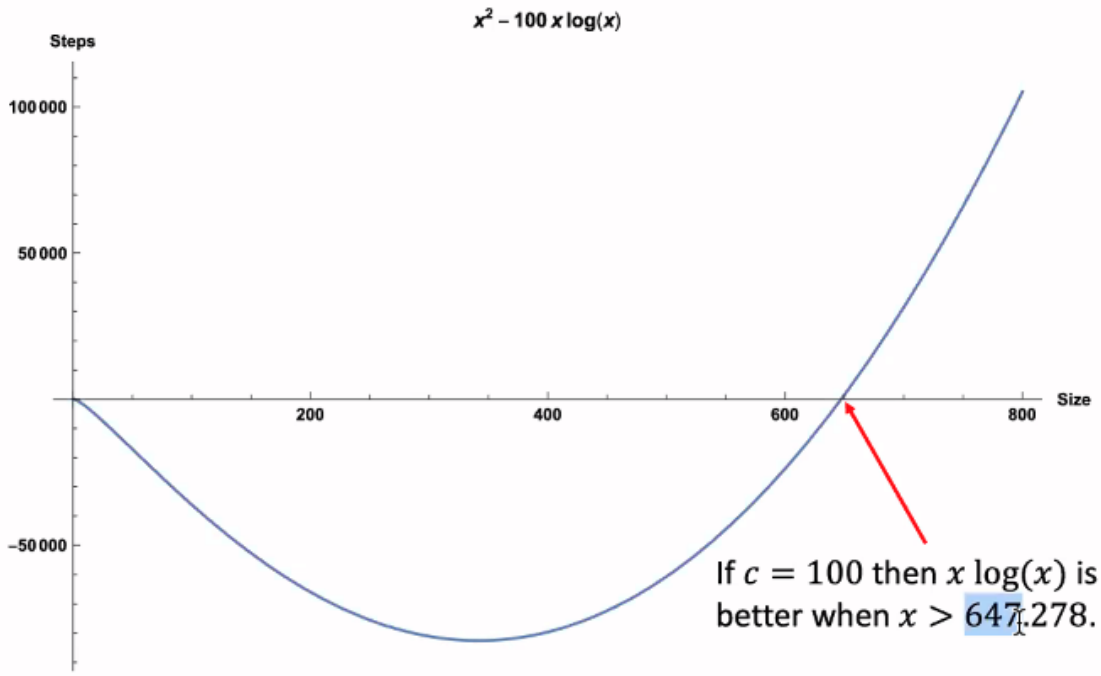


What does that tell me?
It's probably better to use bubble sort for less than 50 items

```c
#include "bubblesort.h"
#include <stdbool.h>
#include <stdint.h>

void bubbleSort(uint32_t a[], int length) {
  bool swapped;
  do {
    swapped = false;
    for (int i = 1; i < length; i += 1) {
      if (a[i - 1] > a[i]) {
        SWAP(a[i - 1], a[i]);
        swapped = true;
      }
    }
    length -= 1;
  } while (swapped);
  return;
}
```

Consider $x^2 - 100\,x\log(x)$



What about recursion O(n)

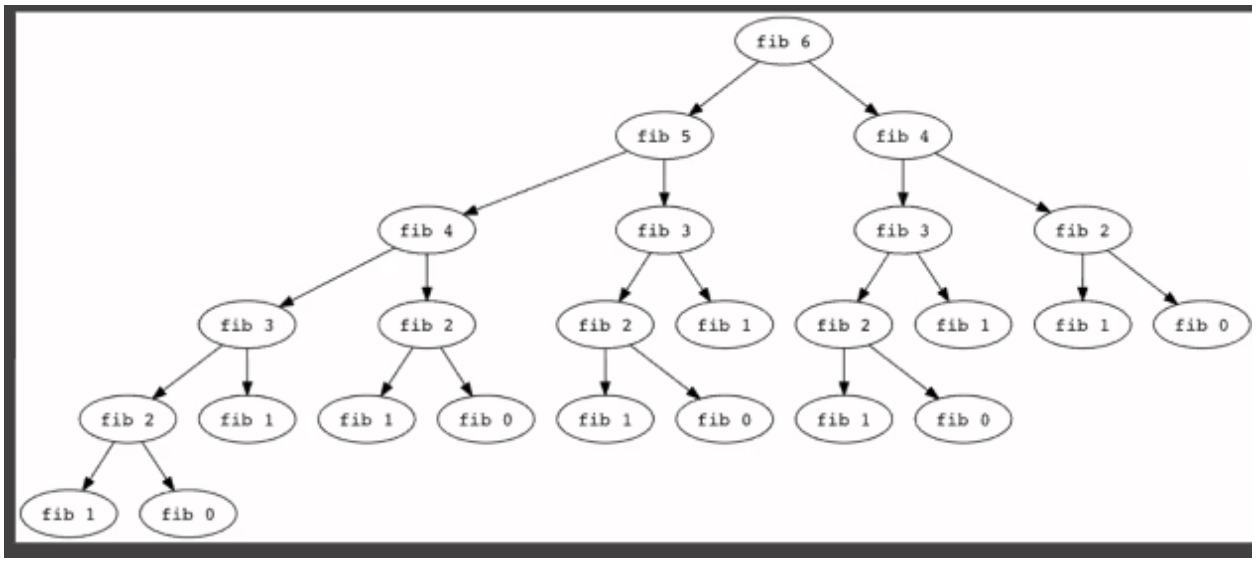
```
int f(uint32_t n) {
  if (n == 0) {
    return 1;
  } else {
    return n * f(n - 1);
  }
}
```

```
int f(int n) {
  if (n == 0 || n == 1) {
    return n;
  } else {
    return f(n - 1) + f(n - 2);
  }
}
```

$$O\left(\left(\frac{1+\sqrt{5}}{2}\right)^{n}\right) < O(2^{n})$$

Recursive Fibonacci
f(0)=0
f(f)=1
f(n)=f(n-2)+f(n-1)

memoization is an optimization technique used primarily to speed up computer programs by storing the results of expensive function calls and returning the cached result when the same inputs occur again.



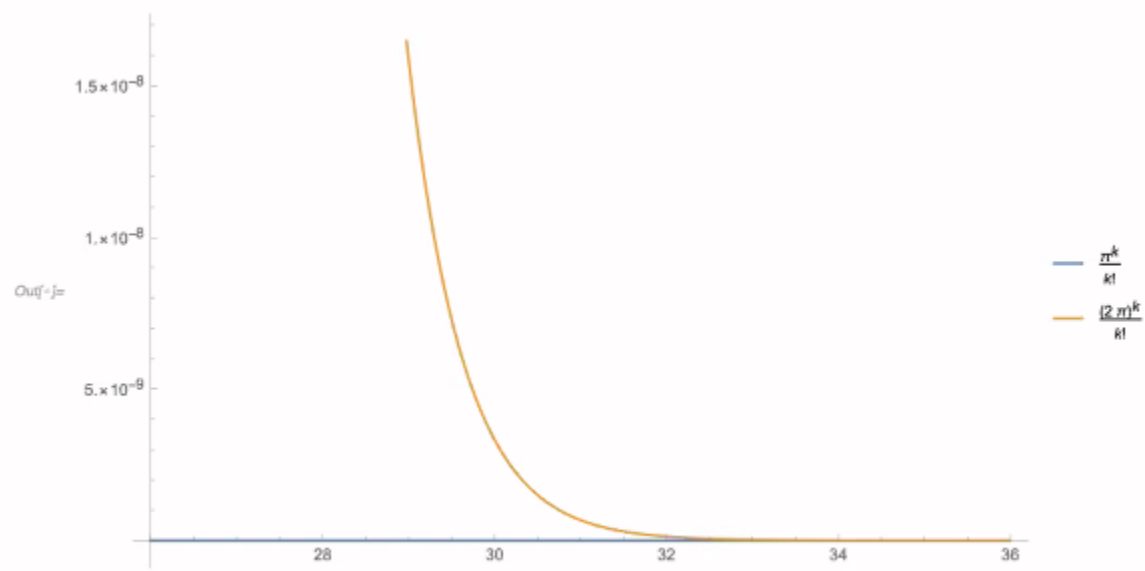
Another algorithm, same value, this time complexity is O(n)

```
int fib(int n) {
  int a = 0, b = 1, sum = 0;
  if (n < 2) {
    return n;
  }
  for (int i = 2; i <= n; i++ 1) {
    sum = a + b;
    a = b;
    b = sum;
  }
  return sum;
}
```

```
double Sin(double) {
    x = modulus(x, 2 * M_PI); // Normalize to [-2π, 2π]
    double sgn = 1, val = x, trm = x;
    for (int k = 3; abs(trm) > epsilon; k += 2) {
        trm = trm * (x * x) / ((k - 1) * k);
        sgn = -sgn;
        val += sgn * trm;
    }
    return val;
}
```

- Error term is $x^k/k!$
- How large does that have to be for it to be less than epsilon?
- The largest x=2pi, so we can figure it out

Plot the ratio $(2pi)^k/k!$



- For 10 digits of accuracy, we will need at most 35 iterations
- We pick the largest x, in this case 2pi

```
long double Sqrt(long double x) {
  long double f = 1.0;
  while (x > 1) {
    x /= 4.0;
    f *= 2.0;
  }
  long double m, l = 0.0, h = (x < 1) ? 1 : x;
  steps = 0;
  do {
    steps += 1;
    m = (l + h) / 2.0;
    if (m * m < x) {
      l = m;
    } else {
      h = m;
    }
  } while (abs(l - h) > epsilon);
  return f * m;
}
```

Extreme example

- Factor:

  $n =$ 3,712,368,003,163,152,165,726,917,914,396,003,989,286, 393,900,857,974,804,551,541

- You could try every number 2, 3, 4, … and see if it divides n.
  - Your first success would be 1,821,640,449,726,328,871,578,165,744,201.
  - Which means you will finish in about 10^{13} years using your laptop.

- We believe integer factorization is hard
- We've known for 2300 years how to find all primes up to n in O(nlog(log(n)) time
- The best we know is the General number field sieve

Public RSA key is the product of two large primes p and q

Need to factor the number n
In factoring a number, the number of binary digits required to express the number you're factoring determines the n

Summary

- Complexity can measure time, space, or both
    - Time is "how many steps it takes"
    - Space is "how much memory it uses"
- The algorithm that you choose has the largest impact on the performance of your program
- For small problems, it may be better to use a worse algorithm if the better algorithm has a larger constant
    - You need to take the time and examine it carefully

4/23/21
Sorting

- Putting things into a defined order
- Dictionaries are sorted in lexicographical order
    - Aka Alphabetical order
- Numbers can be sorted in their natural order, or reverse order
- There are total and partial orderings, but we will only concern ourselves with total orderings

Why do we sort?

- Sorting adds information to our data
- For example, we now can make assertions about before, after, lesser, greater, and so forth
- Here are a few examples:
    - We can search most efficiently in sorted data
    - We can merge sorted lists efficiently
    - We can detect duplicates efficiently

You need a sorted list for binary search for example

- Enumerate all of the possible orderings of n objects
    - There are n! Such orderings
- Pick the one that is in order
- But that is a bad idea

n	$n!$
1	1
10	3628800
20	2432902008176640000
30	2.652×10^{32}
40	8.158×10^{47}
50	3.041×10^{64}
60	8.320×10^{81}

ber exceeds the number of protons in the known universe.

Second idea

- Go through list, find the smallest,
    - Put it in the first slot
- What do I do with what was already there
    - Swap them
- Once the smallest item is found, we never look at it again, in effect we have a smaller by 1 unsorted array
- Repeat process for smaller and smaller subarrays: n-1, n-2,...,1
- This is an instance of selection sort
- (n(n+1))/2
- $O(n^2)$

Minsort

```c
#include "minsort.h"
#include <stdint.h>

// minIndex: find the index of the least element.

int minIndex(uint32_t a[], int first, int last) {
  int min = first;
  for (int i = first; i < last; i += 1) {
    min = a[i] < a[min] ? i : min;
  }
  return min;
}

// minSort: sort by repeatedly finding the least element.

void minSort(uint32_t a[], int length) {
  for (int i = 0; i < length - 1; i += 1) {
    int min = minIndex(a, i, length);
    if (min != i) {
      SWAP(a[min], a[i]);
    }
  }
  return;
}
```

Good use of the ternary operator

7

Third idea

- If we have an array with on element, it's already sorted
- If we have a second element, it must go either before or after the first element
    - Now the two element subarray is sorted
- Given a third element, it must be before the first, between the two, or after the second
    - Now the three elements subarray is sorted
- Proceed to 4,5 elements, So,1,2,3,4..,(n-1) steps
- We only have to consider all of the preceding elements in the worst case
- (n(n-1))/2
- Still $O(n^2)$

Insertionsort

```c
#include "insertionsort.h"
#include <stdint.h>

void insertionSort(uint32_t a[], int length) {
  for (int i = 1; i < length; i += 1) {
    int j = i;
    uint32_t tmp = a[i];
    while (j > 0 && a[j - 1] > tmp) {
      a[j] = a[j - 1];
      j -= 1;
    }
    a[j] = tmp;
  }
  return;
}
```

Shell sort is a variation of an insertion sort

-For insertion sort, you gradually build up a subarray of sorted elements; when you want to insert something in to the sorted subarray, you can optimize by searching with binary search
-Reduces the number of comparisons (c) but the worst case is still $O(n^2)$

Fourth idea

- Suppose we have an unsorted array
- Look at the first two elements, if the first is greater than the second then swap them
- Move on to the second and third, swap if they are out of order
    - Continue on up to the last pair of elements in the array
- The last element of the array is the largest
- bubble Sort

Without the hack!

```python
def bubble(a):
    for i in range(len(a) - 1):
        j = len(a) - 1
        while j > i:
            if a[j] < a[j - 1]:
                a[j], a[j - 1] = a[j - 1], a[j]
            j -= 1
    return a
```

We can do better

- We observe that swapping means that at least two elements were out of reder
- If we examine every pair, and none of them are inverted then the array is sorted
- So the best case is O(n)

```c
#include "bubblesort.h"
#include <stdbool.h>
#include <stdint.h>

void bubbleSort(uint32_t a[], int length) {
  bool swapped;
  do {
    swapped = false;
    for (int i = 1; i < length; i += 1) {
      if (a[i - 1] > a[i]) {
        SWAP(a[i - 1], a[i]);
        swapped = true;
      }
    }
    length -= 1;
  } while (swapped);
  return;
}
```

Is that the best we can do?

- No we can sort in time proportional to nlogn
- If we have an array in disjoint pairs a[0], a[1], then a[2],a[3] etc.
    - We have n/2 such pairs, but we have to look at both so let's call that n
    - We put the elements of every pair in order (length 2) and call that a run
- If you take a pair of runs, each length 2, and merge them into runs of length 4
    - This takes time proportional to n
- How many times can we double 1, logn

Comparative sorting algorithms

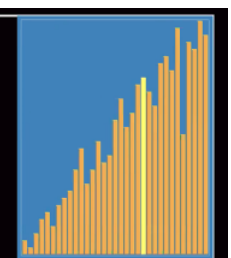
- $O(n^2)$ sorting algorithms
    - Bubble sort
    - Insertion sort
    - Selection sort
    - Quick sort (worst case)
- Shell sort is an $O(n^{5/3})$ sorting algorithm
    - Surprisingly good
    - Complexity of shell sort depends on the gap sequence
- O(nlogn) sorting algorithms
    - Heap sort
    - Merge sort
    - Quick sort (average case)

Comparison of execution times

Base of log doesn't matter for complexity since it's the same as dividing by log(base), just changes constant factor

```
def gap(n):
    while n > 1:
        n = 1 if n <= 2 else 5 * n // 11
        yield(n)

def shellSort(s):
    for step in gap(len(s)):
        for i in range(step, len(s)):
            for j in range(i, step - 1, -step):
                if s[j] < s[j - step]:
                    s[j], s[j - step] = s[j - step], s[j]
    return s
```



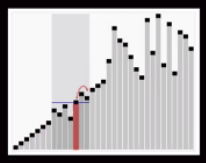
Shell Sort

This Python example uses a generator to create a gap sequence

// is Python's integer division function and does the same thing as floor()

quicksort

```
def QSort(a):
    if len(a) < SMALL:
        return shellSort(a)
    else:
        pivot = (a[0] + a[len(a) / 2] + a[len(a) -1]) / 3
        left  = [ _ for _ in a if  _ <  pivot ]
        mid   = [ _ for _ in a if  _ == pivot ]
        right = [ _ for _ in a if  _ >  pivot ]
        return QSort(left) + mid + QSort(right)
```



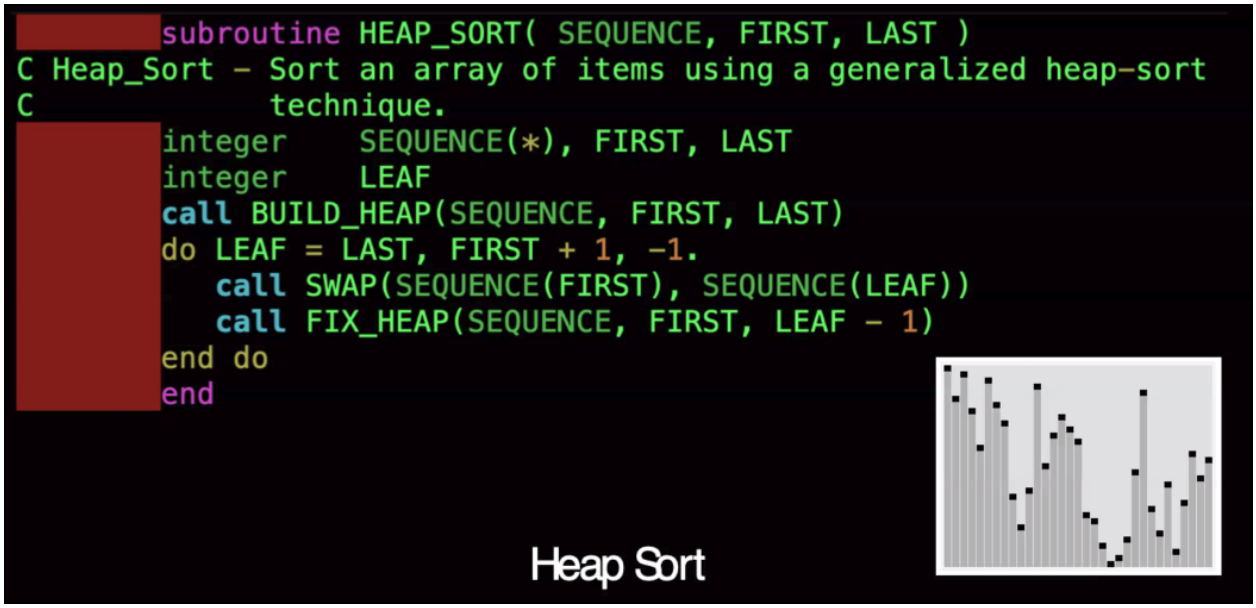
QuickSort

Heaps

- There are types of heaps
    - min/max heaps
    - Leftist
    - Inomial
    - Fibonacci
    - Brodal
    - Radix
    - Uriah

A Max Heap

- A single node is a heap
- It is a heap if the parent is a heap and the trees rooted at both children are heaps
- A parent's value (key) is greater than that of either child
- There is no order among the children



In Fortran

Use the opposite kind of heap for the final ordering you want

Min heap has the smallest value as the root
Max heap has the largest value as the root

Can we do better

- Not using comparisons
- If we make assumptions about the encoding you can use radix sort
    - Runs in time proportional to the number of digit in the key times the number of records
    - Was invented for the mechanical sorting of punched cards
    - For example if we know something about the structure, like it's only integers or 10 digit number (would be 10n or O(n))

IBM punched card

- Was designed with 0 punch in the 0 row
- 1 punch in the 1 row

Bucket sort

- Sort into buckets (ranges) and then sort the buckets
- Imagine if the numbers are integers
    - In a range (max-min) you have memory for
    - Set bucket size of 1
    - Count the number of each value
    - O(n)

Summary

- Sorting is a fundamental operation, so doing it efficiently has a huge impact on computing
- Analysis of algorithm complexity is an important topic; a better algorithm makes a lot more difference than a faster computer
- You will encounter many instances where you need to use a sorting algorithm, and the best one will depend on the circumstances and the data structures employed

In a heterogeneous array in languages other than C, you define how it is sorted eg by byte size; standard library probably wouldn't define how to sort it well